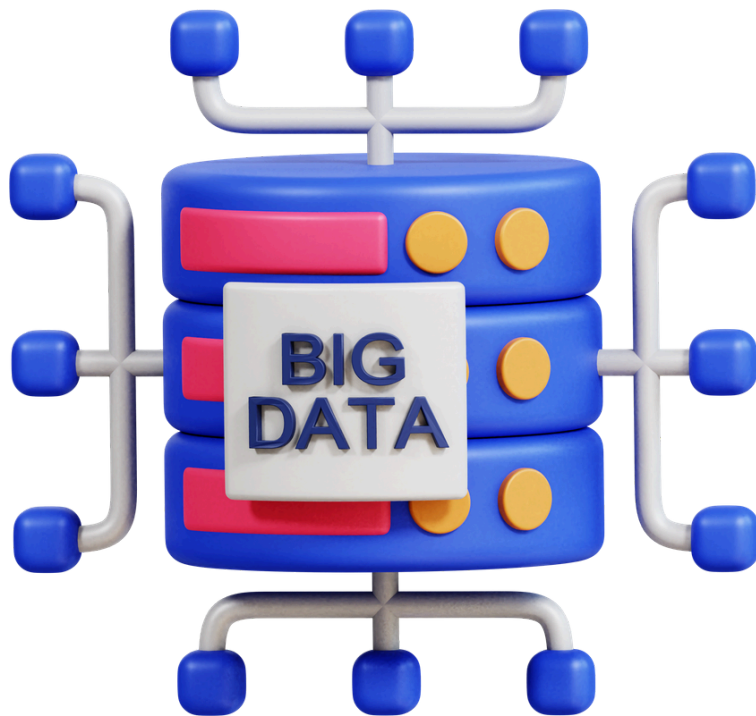

Understanding Spark Memory Management



@Ganesh R



INTRODUCTION

Apache Spark is a powerful, open-source distributed computing system that provides an interface for programming entire clusters with implicit data parallelism and fault tolerance. One of the critical aspects of achieving optimal performance in Spark applications is effective memory management. This section delves into the key components and concepts involved in Spark memory management.

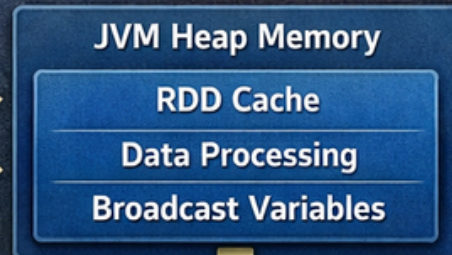
CONTENTS

1. Overview of Spark Memory Management
 - Importance of Memory Management
 - Components of Spark Memory
2. Memory Components in Spark
 - Executor Memory
 - Storage Memory
 - Execution Memory
 - Unified Memory Management
3. Configurations and Parameters
 - Spark Memory Configuration Parameters (calculation)
4. Memory Management Techniques
 - Static Memory Management
 - Dynamic Memory Management
5. Garbage Collection in Spark
 - JVM Garbage Collection
 - Impact on Spark Performance
6. Common Memory Management Issues
 - Out of Memory Errors
 - Memory Leaks
 - Strategies to Mitigate Issues
7. Best Practices for Spark Memory Management

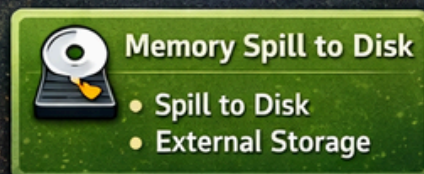
Spark Memory Management



Memory Usage



Memory Management



1. Overview of Spark Memory Management

Memory management in Spark is essential for ensuring efficient resource utilization, minimizing garbage collection overhead, and balancing the needs of various Spark components like caching, shuffling, and computation.

Importance of Memory Management

What is JVM?

- JVM (Java Virtual Machine) is a virtual computer that runs Java programs. It provides an execution environment where Java bytecode is executed.
- When we write code in Python using PySpark, we are actually using a wrapper around Spark's Java/Scala APIs. Internally, Spark is written in Scala, and Scala runs on the JVM. So even if we write PySpark code in Python, the actual execution happens inside the JVM.
- Why JVM is Important in Spark?
- In Spark architecture, the major operations run on On-Heap Memory, which is managed by the JVM.

1. Resource Utilization

Efficient memory management ensures that available resources are used optimally. It prevents scenarios where some tasks are starved of memory while others are over-provisioned.

2. Performance Optimization

Proper memory allocation and management reduce Garbage Collection (GC) overhead.

This leads to better application performance.

3. Reliability

- Effective memory management helps avoid common issues such as OutOfMemory errors, which can cause job failures and reduce overall system reliability.

Key Understanding:

- Spark is written in Scala.
- Scala runs on the JVM.
- PySpark is just a wrapper over Spark's Java/Scala APIs.
- The underlying operations always happen inside the JVM.
- On-Heap memory is managed by the JVM.
- So, even when we write PySpark code in Python, execution happens inside the Java Virtual Machine (JVM).

2. Memory Components in Spark

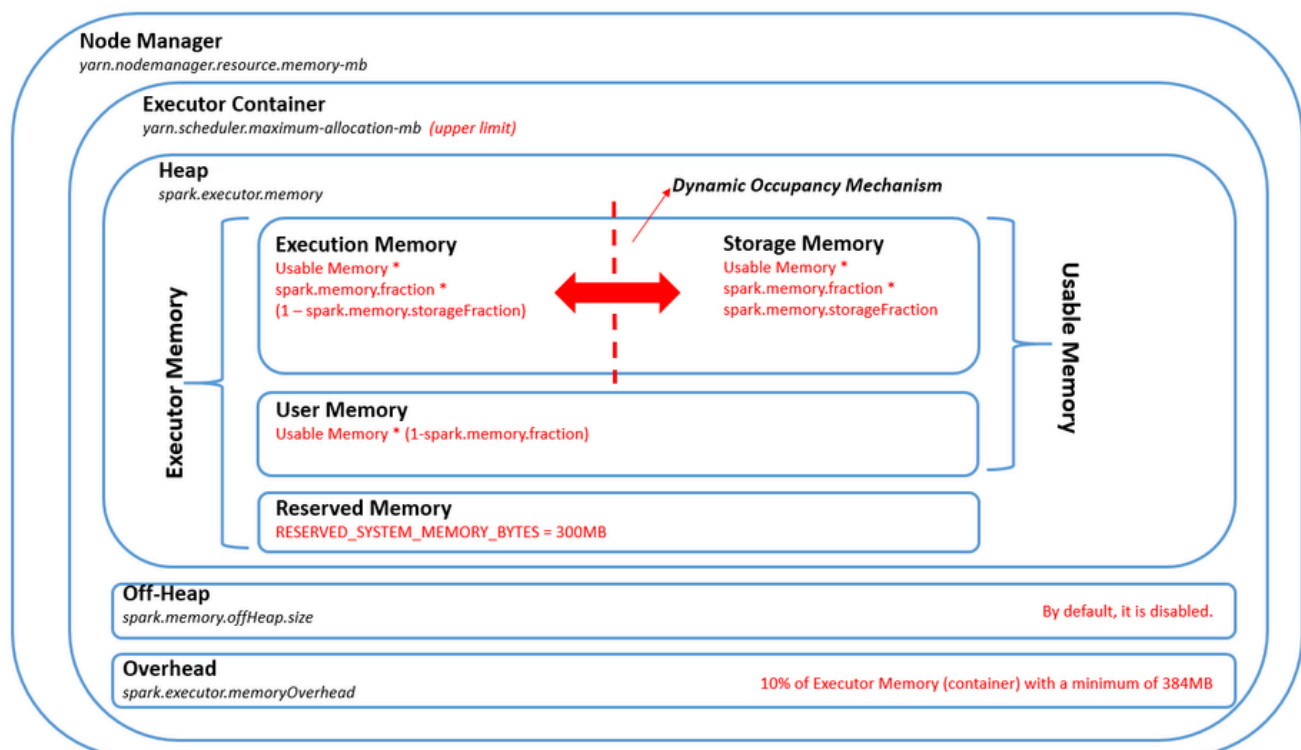
On-Heap Memory is divided into the 4 sections

1. Execution memory: it is the place where all of the joins, shuffles, sorting and group By take place.

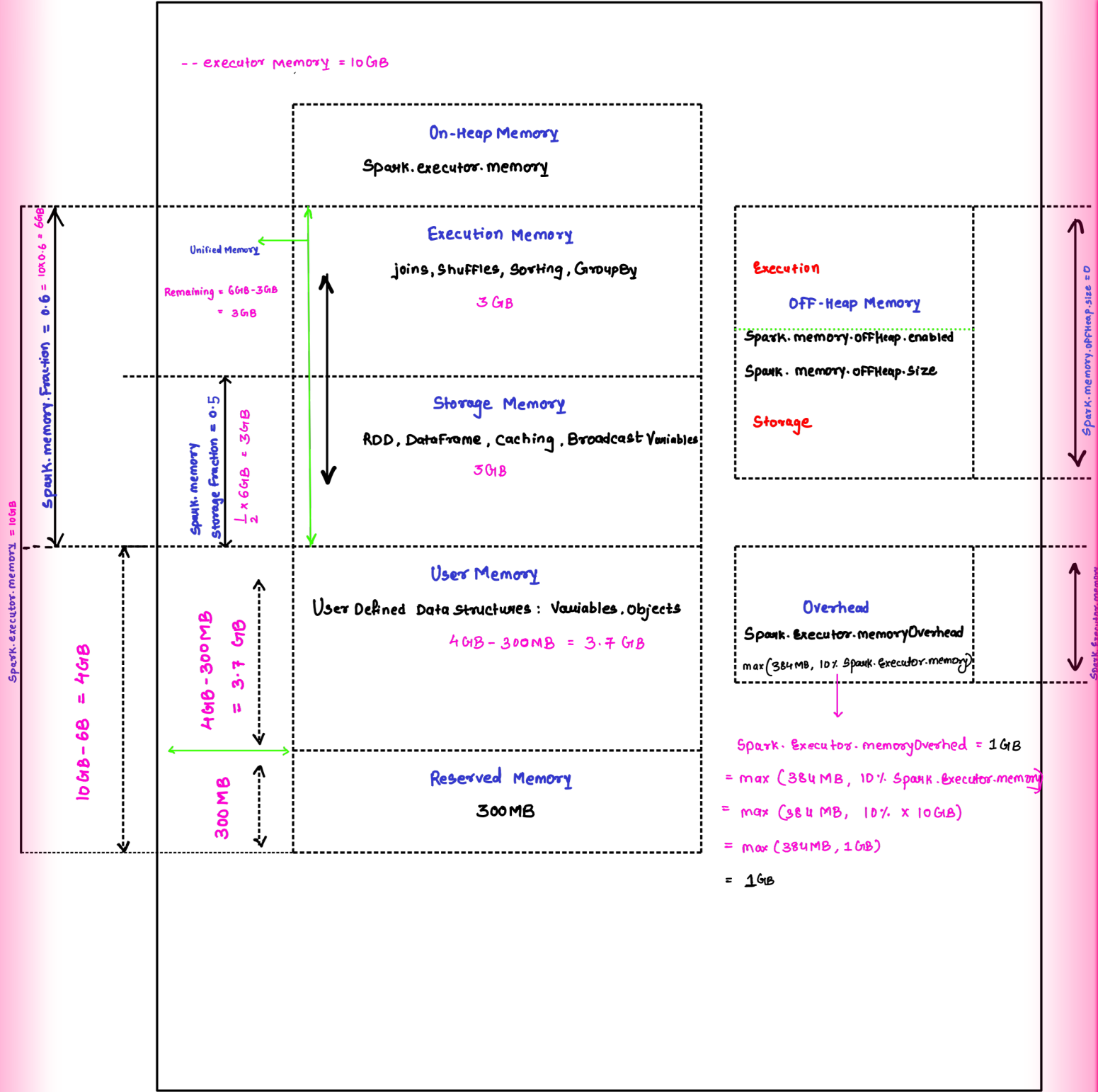
2. Storage memory : it is the place where caching take place and this caching can be on RDD or Dataframes and it also store the broadcast variables. Unified memory : Execution memory + Storage memory

3. User memory : it is use to store the user object for example : variables , list, sets, dictionary that we define and even for storing UDF(USER DEFINED FUNCTIONS)

4. Reserved memory : this spark needs for running itself. And for storing internal object. Overhead memory is used for some internal system level operation. Off-Heap memory is used to reduce the garbage collection



3. Configuration and Parameters



Calculation Configuration and calculations

When we run our spark Job we specify:

- - executor memory : **10 GB**

OnHeapMemory calculation

Spark.memory.fraction = **10GB * 0.6 = 6GB**

Spark.memory.storagefraction = **6GB * 0.5 = 3GB. (STORAGE MEMORY)**

Spark.memory.executionmemory = **6GB - 3GB = 3GB (EXECUTION MEMORY)**

Overhead memory (max 384 or 10% of spark.executor.memory) = **MAX (384 MB OR 10% * 10GB) = 1GB**

OFFHeap Memory calculation

Spark.memory.offHeap.size = 0 But we can set this to non-zero number and allocate memory but the way off-heap memory is structured this also has 2 parts like execution and storage so it looks the same as the unified memory. Normally it is disabled but we can enable it by setting the parameter shown in the diagram. In starting we can provide like 10% to 20% of the execution memory

Spark.memeory.offHeapenabled = true

Spark.memory.offHeap.size = 10GB * 10% = 1GB

Questions:

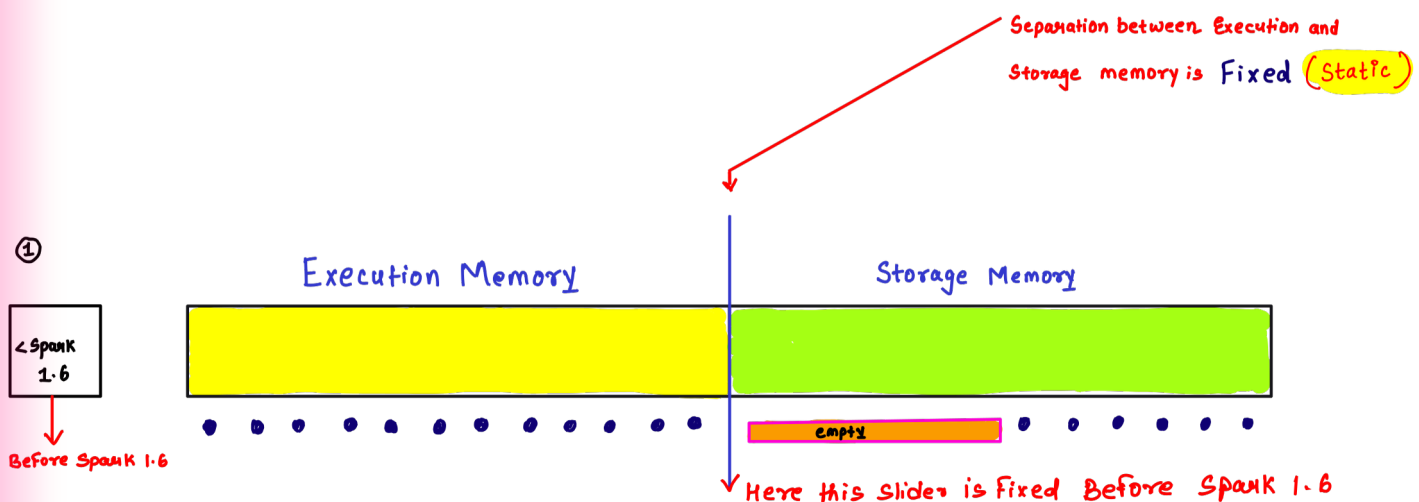
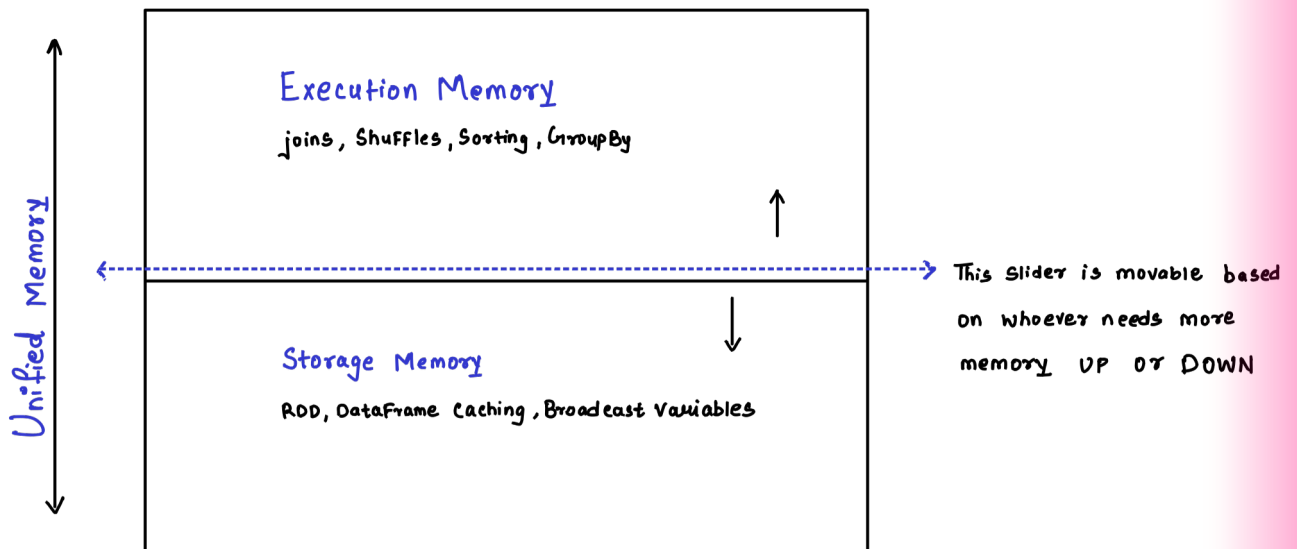
If 10GB of executor memory is allocated only to the On-Heap memory then what memory will be allocated to Off-Heap Memory? So important thing to note here is executor memory is allocated to On-Heap memory but when spark request to the YARN cluster manager for the memory then spark will add on the executor memory + overhead memory + Off-Heap Memory (if enabled) to requested YARN.

In the above case let's assume that Off-Heap Memory is disabled then YARN cluster manager will take total of executor memory + overhead memory = 10GB + 1GB = 11GB will be requested for the container.

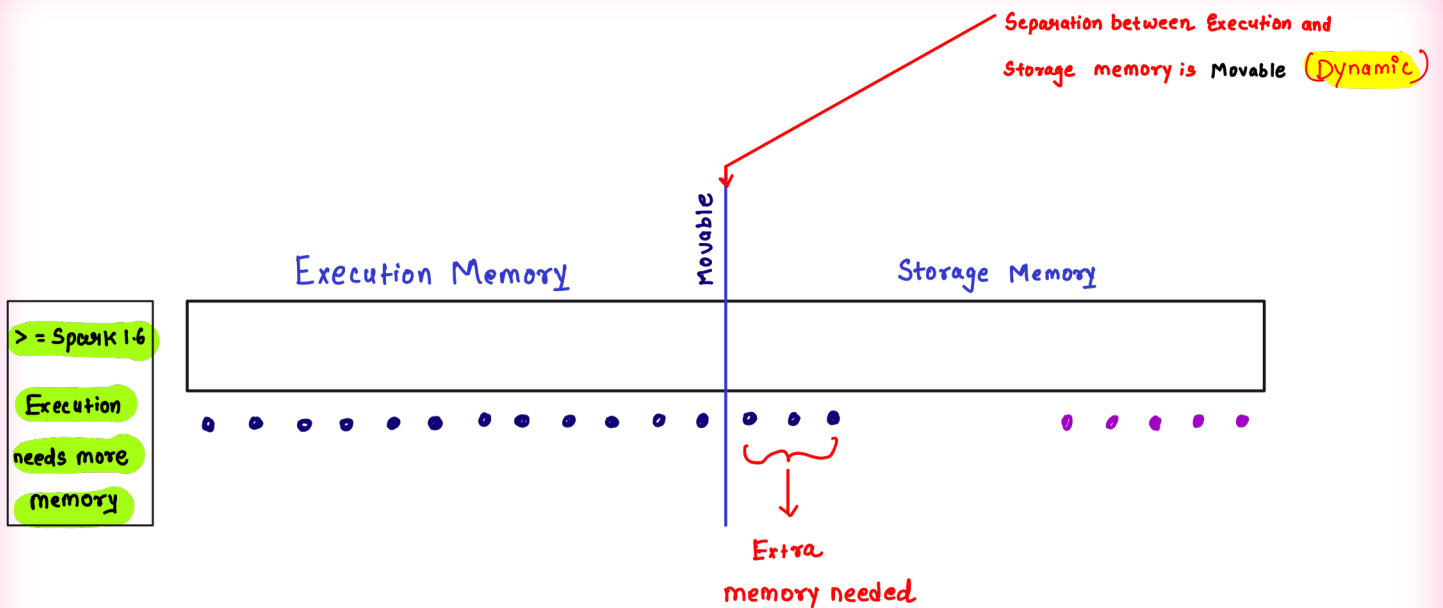
4. Memory Management Techniques

understand unified memory in detail:

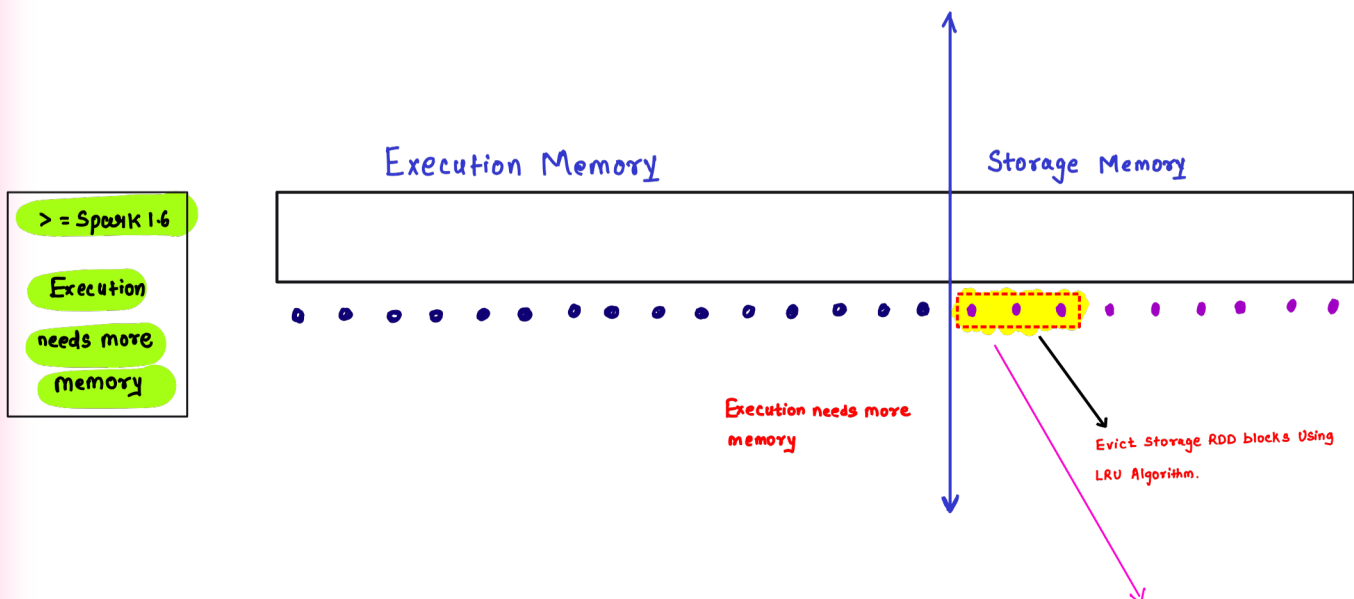
Together of execution memory and storage memory is unified memory because of the dynamic memory management strategy of the spark. Which means if Execution needs more memory then it will simply use some of the storage memory. And if storage wanted more memory then it will simply go ahead and use Execution memory. But there is a priority which is given to the Execution memory because all of the joins, shuffle, sorting and group by all this happens here.



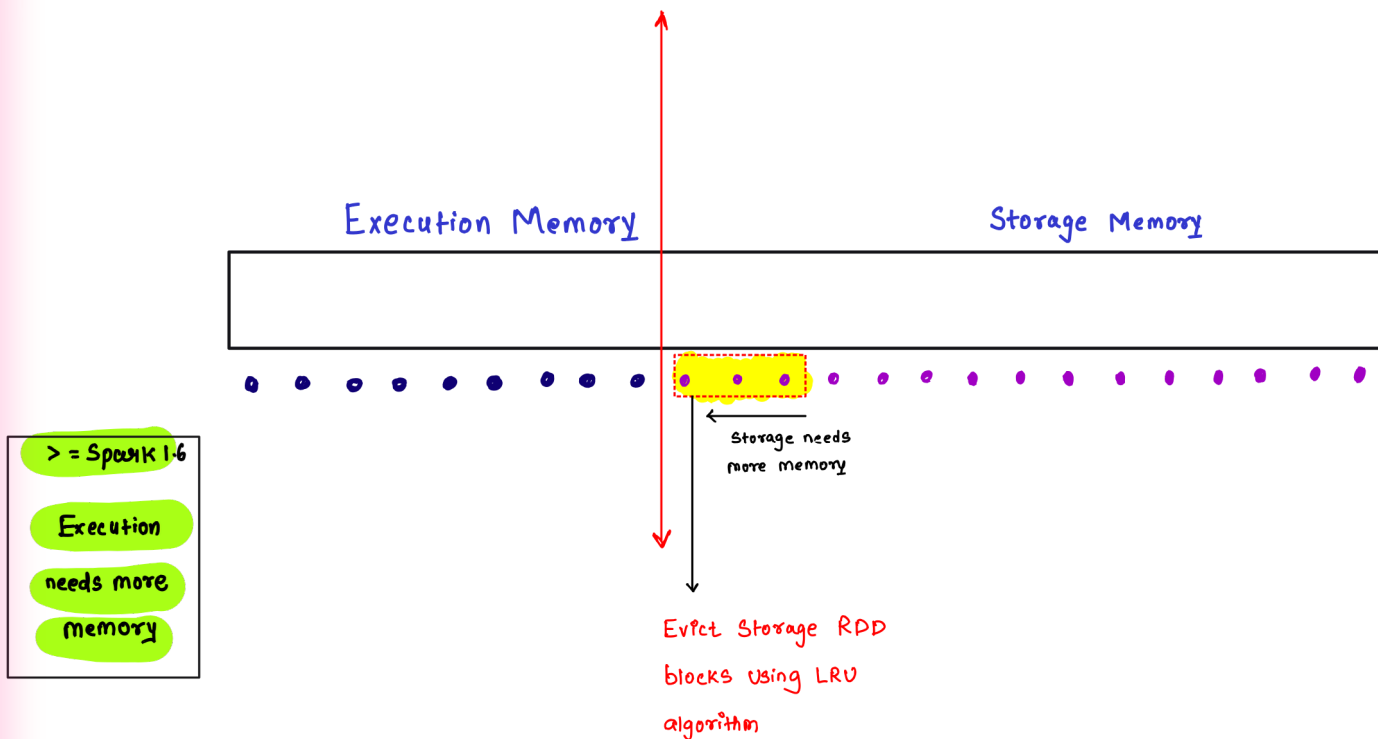
So, if Execution Memory is full and if it needs more memory despite Storage being empty, it won't be able to use the empty part of the Storage Memory. So, this is the big waste of Memory available.



Let's say execution needs more memory and there is a vacant memory and storage is not using that then in that case we can simply go and use that portion of storage memory.



In this case as the execution needs more memory execution has occupied some part of the storage memory for more joins or shuffles or aggregation etc so in this case storage will go to evict some of its block and give it to execution memory and this eviction will take place using LRU algorithm which LEAST RECENTLY USED BLOCKS will be evicted. So this space now will be going to be available for execution memory.



In this case storage has occupied lot of memory for caching, broadcasting purpose and now Execution memory also needs memory and **execution memory** has **priority** because all of the aggregation , joins and shuffle operation takes place in the execution memory. so in that case it will tell storage memory to evict the part of its memory in this case storage will evict memories based on LRU (least recently used block).

So now we know that majority of the spark operation takes place in On-Heap Memory all types of joins, aggregation, shuffles and sorting and this On-Heap Memory is managed by the JVM

5. GARBAGE COLLECTION

- Garbage Collection (GC) runs periodically in the JVM to clean up unused objects from On-Heap memory.
- However, these GC cycles pause application execution (known as stop-the-world pauses), which can negatively impact Spark job performance — especially in large-scale or memory-intensive workloads.



On-Heap vs Off-Heap Memory

◆ **On-Heap Memory**

- Managed by the JVM
- Closest to Spark execution
- Faster access compared to off-heap
- Subject to Garbage Collection
- When full → triggers GC cycle → pauses execution

If On-Heap memory becomes full:

- GC pauses the current program execution.
- Cleans up unused objects.
- Frees space.
- Resumes execution.
- Frequent GC cycles can significantly reduce performance.

◆ **Off-Heap Memory**

- Managed by the Operating System, not the JVM
- Not subject to JVM Garbage Collection
- Slightly slower than On-Heap memory
- Helps reduce GC overhead
- Can reduce memory spills to disk

If Spark has two options:

- Spill data to disk (slowest)
- Use Off-Heap memory (faster than disk)

👉 Using Off-Heap memory is usually the better option than spilling to disk.

⚠️ Why Off-Heap Memory Must Be Used Carefully

- Since Off-Heap memory is not managed by the JVM:
- Developers must handle allocation and deallocation carefully.
- Improper handling can lead to native memory leaks.
- It adds complexity to memory management.
- Spark developers consider Off-Heap memory to:
- Reduce GC pauses
- Minimize disk spills
- Improve performance in memory-heavy workloads
- But it should be enabled and tuned with caution.

🎯 Key Takeaway

- On-Heap = Fast but GC overhead
- Off-Heap = Avoids GC pauses but requires careful management
- Disk Spill = Slowest option

In high-performance Spark workloads, properly tuned Off-Heap memory can significantly improve execution efficiency.

6. Common Memory Management Issues

Out of Memory (OOM) Errors

- Out of Memory errors occur when Spark tasks require more memory than what is allocated or available.

Common Causes:

- Improper Spark or JVM configuration
- Processing very large datasets
- Wide transformations causing heavy shuffling
- Inefficient code (e.g., excessive caching, collect() misuse)

Impact:

- Job failures
- Executor crashes
- Performance degradation

Memory Leaks

- Memory leaks happen when objects are not properly released after use, leading to gradual memory consumption over time.

Common Causes:

- Caching data without unpersisting
- Accumulating objects in driver memory
- Long-running streaming applications
- Improper broadcast variable handling

Impact:

- Increasing memory usage over time
- Frequent garbage collection
- Eventual executor or driver failure

Strategies to Mitigate Memory Issues

✓ **Use Efficient Data Serialization**

- Choose serialization formats like Kryo to reduce memory footprint and improve processing speed.

✓ **Optimize Spark Configurations**

- Tune parameters such as:
- `spark.executor.memory`
- `spark.driver.memory`
- `spark.memory.fraction`
- `spark.sql.shuffle.partitions`

✓ **Monitor and Profile Regularly**

- Use Spark UI, logs, and monitoring tools to:
- Track memory consumption
- Analyze GC behavior
- Identify skew and shuffle bottlenecks

✓ **Write Memory-Efficient Code**

- Avoid unnecessary wide transformations
- Use `reduceByKey` instead of `groupByKey`
- Avoid excessive `collect()`
- Unpersist cached data when no longer needed

7. Best Practices for Spark Memory Management

Monitor and Profile

- Use Spark's built-in tools like the Spark UI and external monitoring systems to track memory usage, analyze execution plans, and identify performance bottlenecks early.

Tune Spark & JVM Parameters

- Adjust Spark and JVM configurations based on workload characteristics and observed metrics. Key configurations include executor memory, driver memory, memory fractions, and garbage collection settings.

Write Efficient Code

- Avoid unnecessary wide transformations (like `groupByKey`, `repartition`) that cause excessive shuffling.
- Use actions judiciously and prefer optimized transformations such as `reduceByKey` over `groupByKey`.

Optimize Data Serialization

- Choose efficient serializers such as Kryo serialization instead of Java serialization to reduce memory footprint and improve performance.

Cache Strategically

- Cache only required DataFrames/RDDs.
- Select appropriate storage levels (`MEMORY_ONLY`, `MEMORY_AND_DISK`, etc.) to balance memory consumption and execution speed.

Thank you

If You Found This
Helpful?

